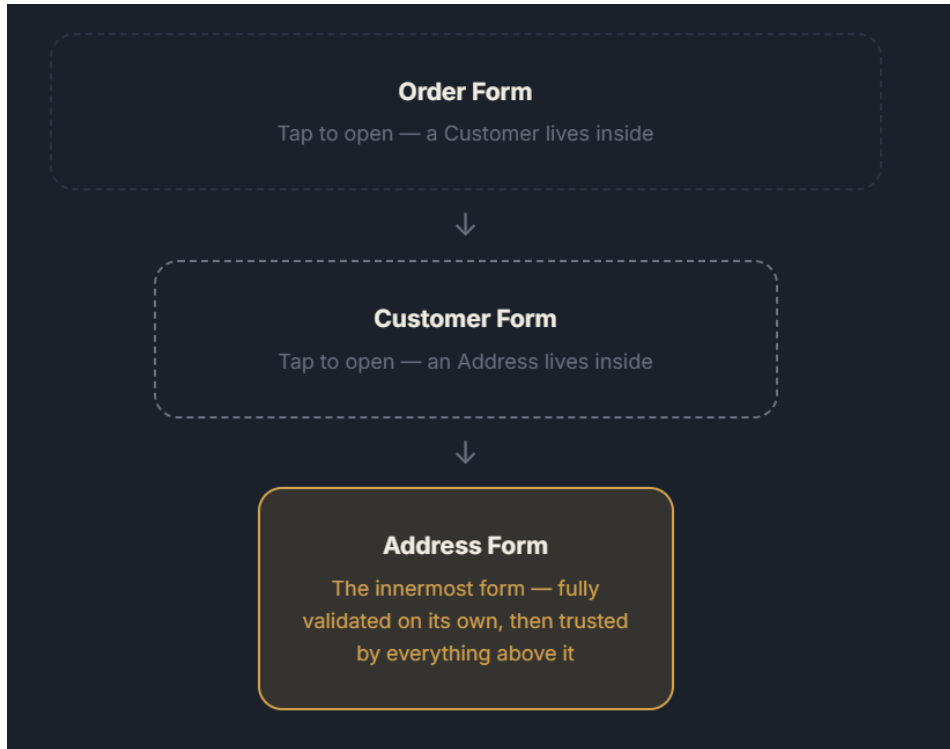


Section 4 — Complex & Nested Data

Real data is rarely flat. A field might hold a list of items, or a smaller form might live entirely inside a bigger one — and that smaller form can even contain another copy of itself, layer after layer, like a folder inside a folder.



Instead of accepting any text, a field can be limited to a short, fixed list of allowed values — like a dropdown. Tap an option below.

- "starter" is on the allowed list, so it passes straight through.
- "main course" is on the allowed list, so it passes straight through.
- "dessert" is on the allowed list, so it passes straight through.
- "anything else" is not one of the fixed choices, so the checkpoint rejects it immediately — no guessing allowed.

Concept Notes

Real-world data is rarely flat. A field may contain multiple values, such as a list of items, or one model may exist inside another model. Pydantic supports complex structures and validates every level of the data.

List and Collection Fields

A model field can contain collections such as lists, dictionaries, sets, and tuples. Pydantic checks both the collection and the values stored inside it.

For example, a list of marks can require every item to be an integer.

Nested Models

A Pydantic model can be used as a field inside another model. For example, a User model can contain an Address model.

The nested model validates its own data before the complete parent model is accepted.

Multi-Level Nesting

Nested models can contain other nested models.

For example:

Order



Customer



Address

Each level is validated according to its own model structure.

List of Nested Models

A model can contain a list of other model objects. For example, an Order model can contain multiple Product models.

Each product in the list is validated separately.

Enum

An Enum is used when a field should accept only a fixed set of values.

For example, an order status may only be:

- pending
- processing
- shipped
- delivered

Any value outside the allowed choices is rejected.

Important Points

- Pydantic supports lists, dictionaries, sets, and tuples.
- Collection items are also validated.
- One Pydantic model can be nested inside another model.
- Nested models make complex data easier to organize and reuse.
- Models can have multiple levels of nesting.
- A field can contain a list of nested model objects.
- Invalid data inside a nested model can cause the parent model validation to fail.
- Enum is used for a fixed set of allowed choices.

Syntax

Collection Field

field_name: list[item_type]

Nested Model

Child Model



Used as a field type



Parent Model

List of Nested Models

items: list[ChildModel]

Enum

Create Enum



Define allowed values



Use Enum as field type